Basic info about the course

- Aim: To teach basic programming skills
- Course web site:
  http://www.onlinesorular.com
- There is no need to buy any Text Book
- (But you can follow

C Programming Language, 2nd Edition by Brian W. Kernighan, Dennis M. Ritchie)

- Slides will be shared through the course website
- 5 hours of class every week
  - 3 hours lecture (lectured by the instructor)
  - 2 hours lab (taught by an assistant)

Basic info about the course

- You will have
  - 2 midterms
  - 1 final

  throughout the semester.
- There might be questions closely related with the labs in every exam.
- You are going to have enough questions on the server

Grading Policy

- Midterm-I 30%
- Midterm-II 30%
- Final 40%

- No code sharing, Similarity check like MOSS will be applied
- According to Marmara University regulations, you must take at least 35/100 points in the final exam and your overall average should be >= 35/100 in order to pass this course

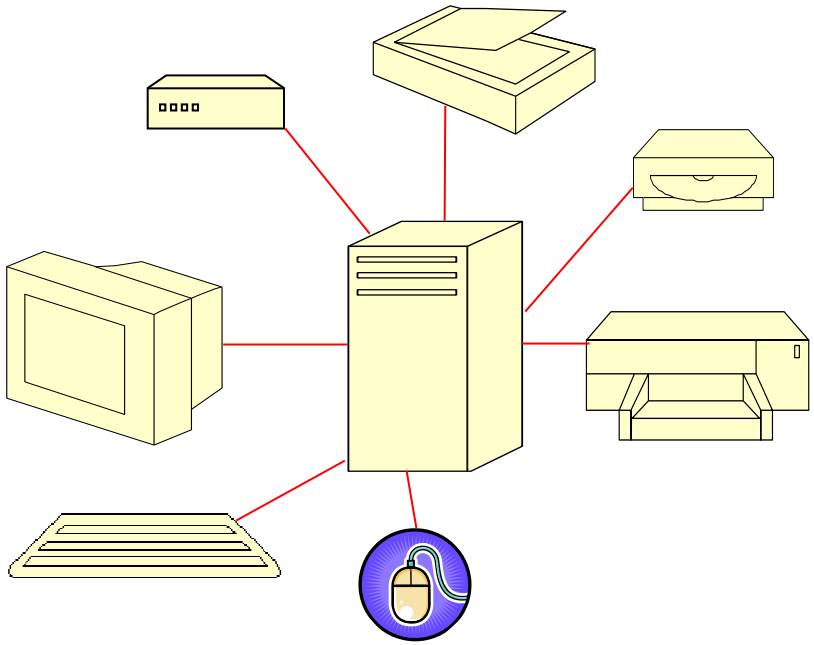
Exams and labs

- Both quizzes and exams will be held on computers.
- Results will be announced just after the quiz
- You can review your paper a few minutes after the end of the quiz.
- Attendance taken in both classes and labs.
  - 70% attendance for the lectures
  - 80% attendance for the labs

Course outline

1- Introduction, printf, scanf, variables, operators, constants
2- Data types, assignment type conversions, type casting, post/pre-increment, type casting
3- If, nested if, logical operators, switch
4- While, for, do-while
5- Nested loops, break, continue
6- Functions, scope, macro-substitution
7- Pointers, variable parameters (aka. call by reference or pointers as function arguments)
8- Arrays, passing arrays to functions, sorting and binary search
9- Strings
10- Multi-dimensional arrays
11- Structs
12- Review

A computer system

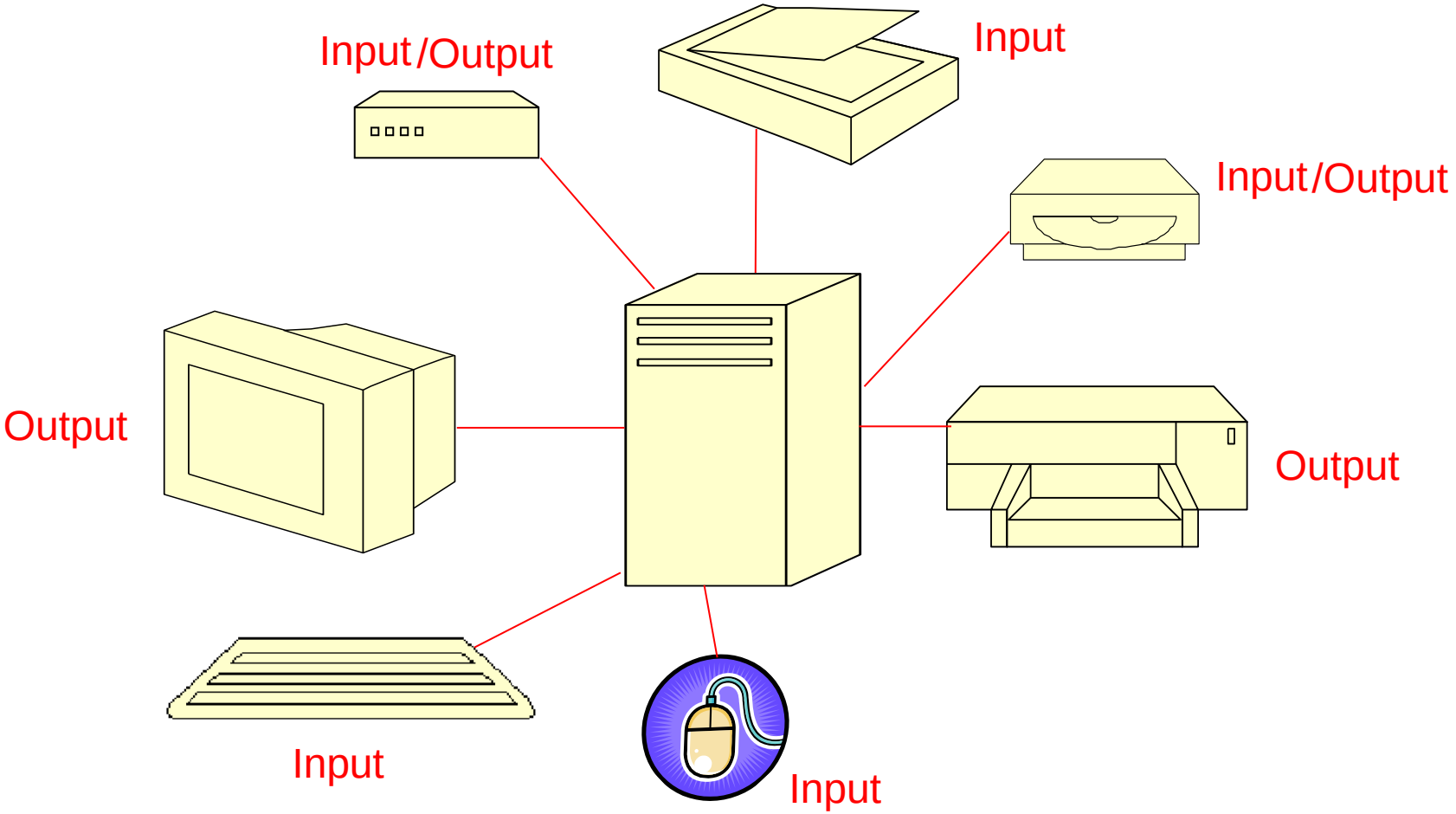


A computer system is composed of:

- a monitor,
- a keyboard,
- a mouse,
- and a case (that contains several controlling components such as processor and alike),
- and also other peripherals like
  - CD player (might have been included in the case),
  - printer,
  - scanner,
  - modem,
  - etc.

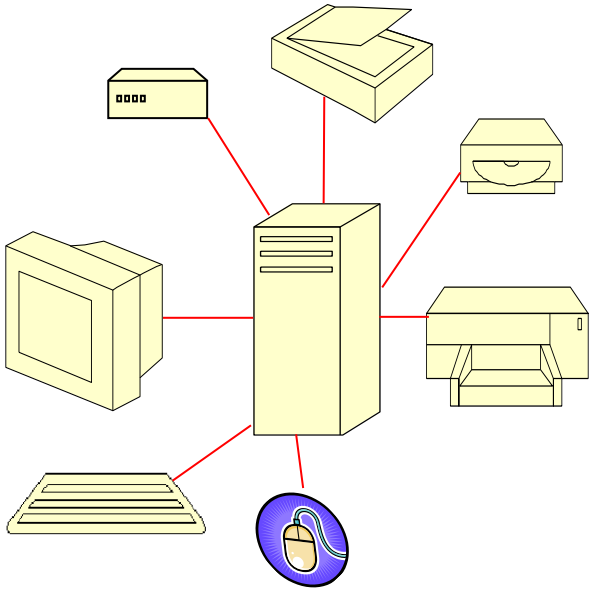
all connected together

Input and output devices



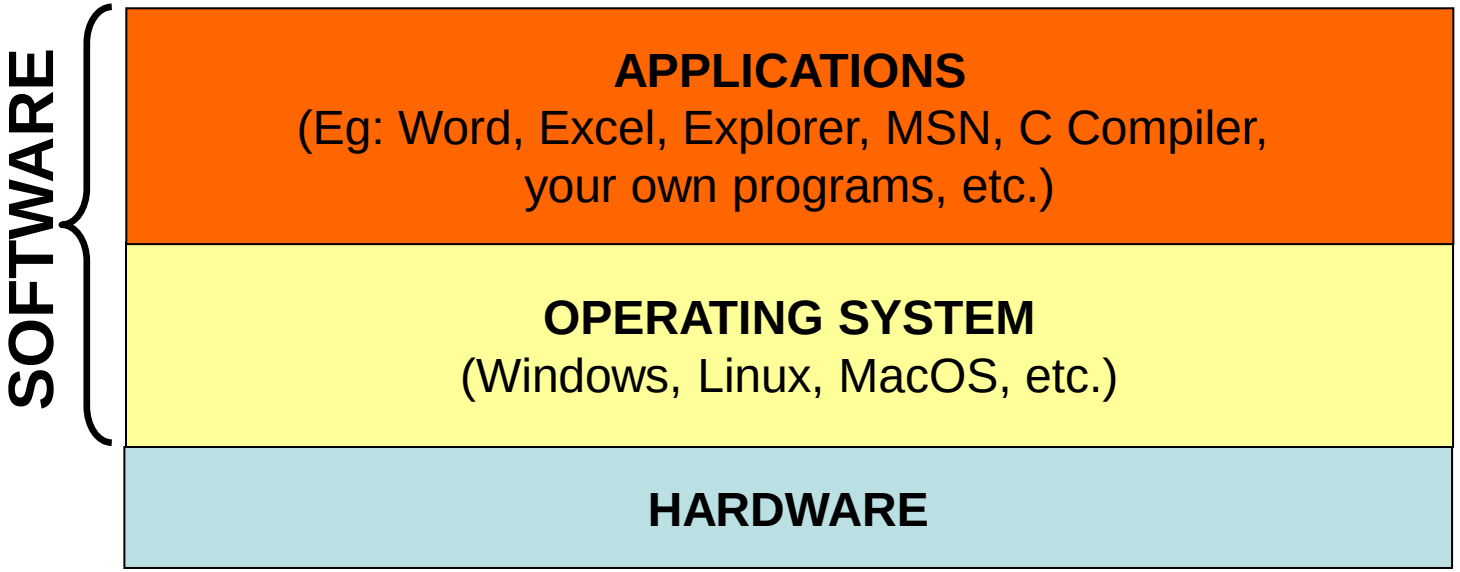
A computer system

- Note that everything could be packed in a single box, but the concepts are the same.



A computer system

- Everything we had in the previous slide is hardware.
  - i.e., physical components that implement what is requested by the software.



A computer system

- In this course, we will learn how to develop our own software (using C language), but we need to understand how our programs will be executed by the hardware.

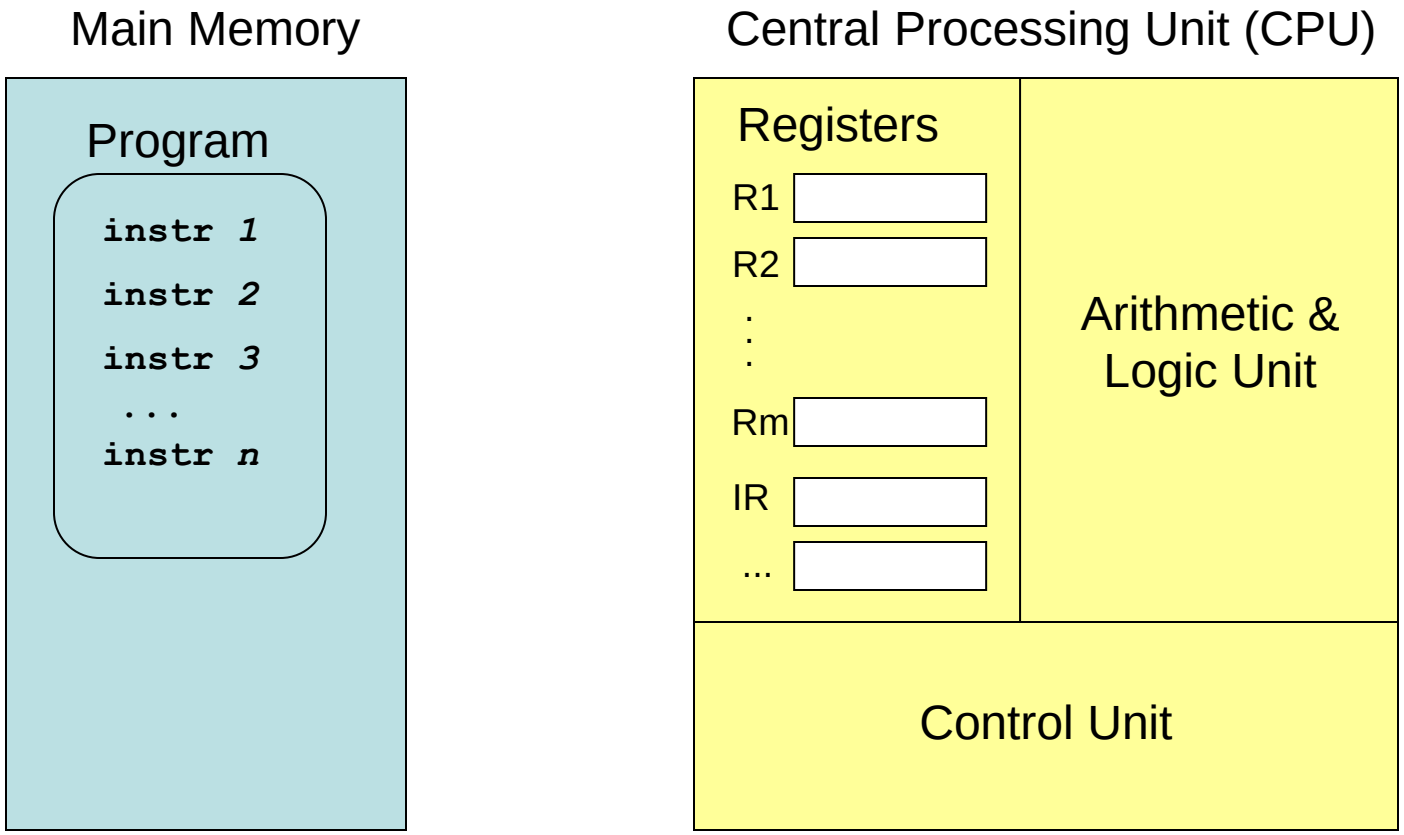
CPU: Central Processing Unit

- In terms of hardware, what is important for us is the CPU.
- It does all processing and control.
  - Everything is controlled and executed by the CPU.

CPU: Central Processing Unit



How are the instructions executed?



How do we write programs?

We write our programs in *"C language"* (which is an English-like language)

We use a compiler (such as GCC, Visual C, Borland C, etc.) to translate our program from "C language" to *"machine language"*

This is the executable code in "machine language."

This is the only thing the computer can understand and run (execute).

```
#include <stdio.h>
int main()
{
   printf("Hello world!");
   return 0;
}
```

(source code)

Compile & Link

```
11101010110010010100010100100001001010100101010100100010
```

(object code)

(machine code or executable code)

Statement vs. Instruction

- Our source code (in C) is composed of statements.
  - Eg: `a=b+c/2;`
- The corresponding machine code is composed of instructions.
  - Eg: `1101001010110010 (divide c by 2)`
    `0110100100100101 (add it to b)`
    `1010110110111011 (put the result in a)`
  - CPU is capable of executing instructions, not statements. Statements may be too complex.
  - Compiler *implements* each statement using several instructions.
    - Eg: The statement "`a=b+c/2;`" can be implemented as

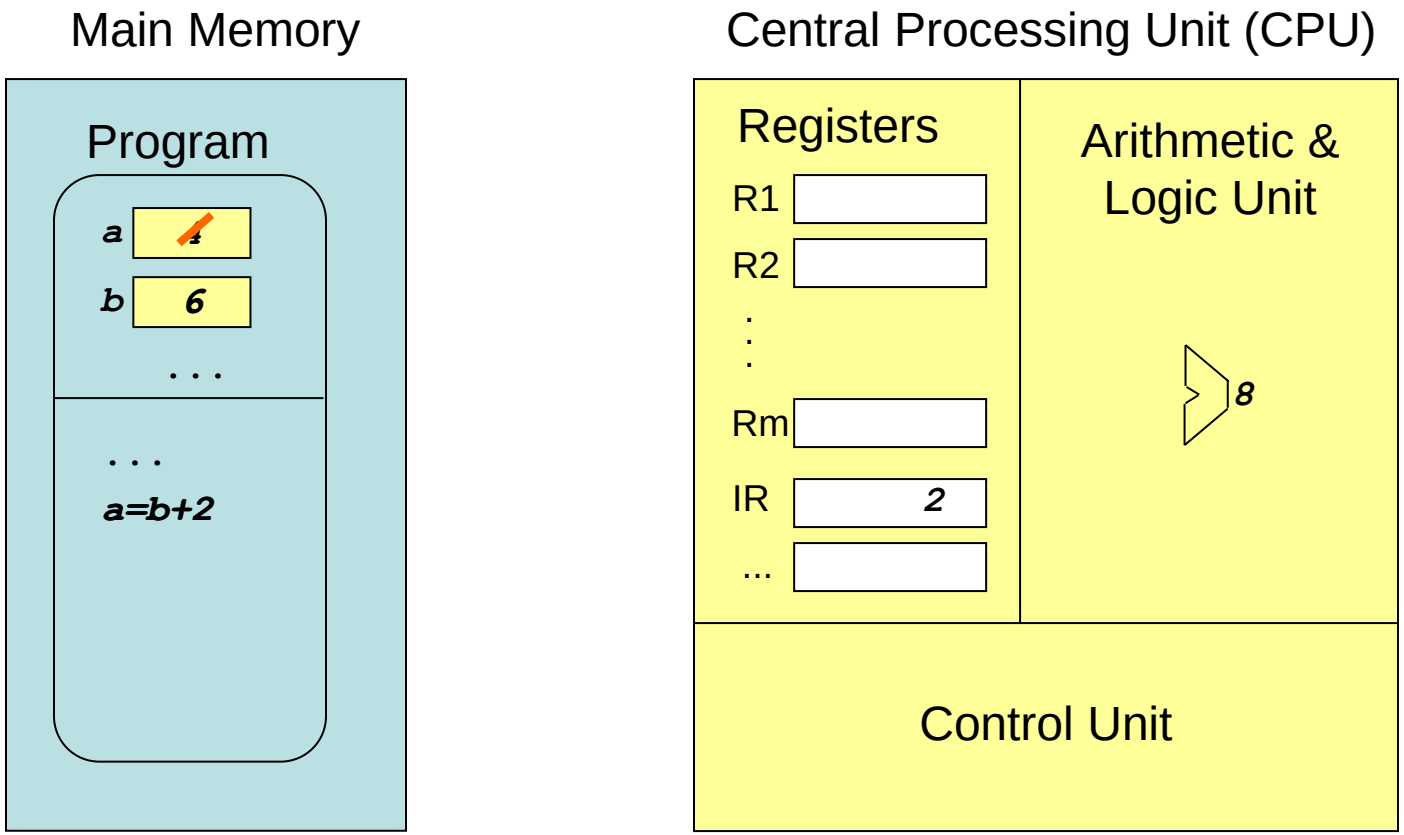
```
temp1 = c/2
a = b + temp1
```

Why have input/output?

- A program should not always produce the same output.
  - O/w, you may keep the result and delete the program after you run it for the first time.
- A program should be consistent; i.e., it should not produce random results.
- Therefore, a program should take some input, process it, and produce some output as the result of that input.

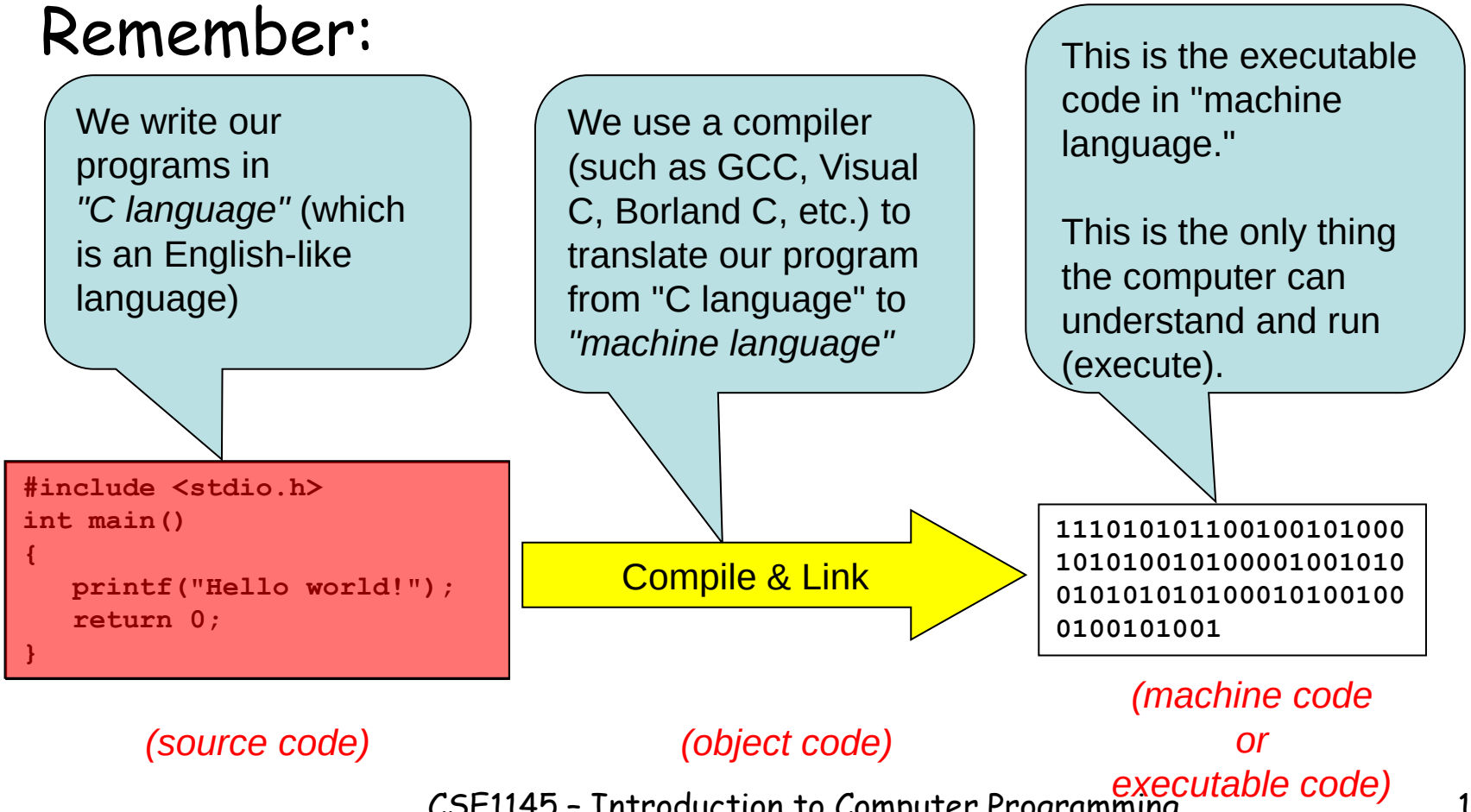
Execution of an instruction

- Let's see how an instruction like "**a=b+2**" is executed.
  - Assume initially **a** is **4** and **b** is **6**.



Welcome to C Programming Language

- Now that we have an overall understanding of the computer, we can start writing C programs.
- Remember:



Our first C program: Hello World

- Every C program has a **`main()`** function.
  - It wraps all the statements to be executed.
- We make use of previously written functions. They are provided by header files.
  - Typically, we include the *standard input/output header* file, named **`stdio.h`**.
- We write all statements inside the **`main()`** function.

```
#include <stdio.h>

int main()
{
   printf("Hello world");
   return 0;
}
```

Need for input

- Note that the Hello World program has no input.
  - Therefore, it always produces the same output:

    `Hello World`

  - So, after we run this program once, we know what it will always produce. Therefore, we don't need the program anymore; we can safely delete it.
- Definitely this is not what we want. (O/w, nobody will pay us ☺)
  - We want to write programs that can take input and produce different results according to the input.
- (Details of I/O functions will be covered in the labs.)

A program that also performs input

C Program

```
#include <stdio.h>

int main()
{
    int a, b, c;

    printf("Enter two numbers: ");
    scanf("%d%d", &a, &b);
    c=a+b;
    printf("Result is %d\n", c);
    return 0;
}
```

Read two integers (decimals) into variables a and b

Display the value of variable c after the text "Result is"

User screen

```
Enter two numbers: 5 8
Result is 13 _
_
```

Variables

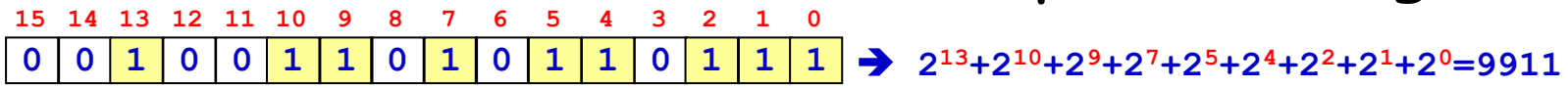
- Operations (such as addition, subtraction, etc.) operate on operands.
- You need some space to store the value of each operand.
- A variable provides storage space for a value.

Variables

- **IMPORTANT:** The value of a variable can never be empty. The value is represented via multiple bits, each of which is either 0 or 1. So, the variable always has a value.
- When a local variable is defined, its initial value is undefined. In other words, it has an arbitrary value. (For the moment, we will not use global variables.)
- So, make sure that the variable has a valid value before you perform any operation based on that value.

Variables

- Each variable consists of multiple bits. E.g.:



- Thus, every value is actually stored as a sequence of bits (1s and 0s) in the computer.
- The number of bits is called the size of the variable.
- The size of a variable depends on the type of the variable, the hardware, the operating system, and the compiler used.
  - So, in your programs NEVER make assumptions about the size of a variable.
  - The size may change due to the factors listed above, and your program will not work.

Variables

```
#include <stdio.h>

int main
{
   int a, b, c;

   a=10;
   b=3;
   c=a-b;
   a=b+2;
}
```

